

List, Array Resizing and Amortized Algorithm Analysis

Jianguo Lu

October 2, 2025

Overview

List Implementation Using Array

Resize array

List Implementation Using Array

Resize array

What is List

- ▶ Array: access elements by index. $a[i]$
- ▶ LinkedList, Stack, Queue: restricted access, using `next()`, `top()`, `tail()` etc.
- ▶ Can we have the best from both?

List defined by ADT

The List ADT

- `size( )` : Returns the number of elements in the list.
- `isEmpty( )` : Returns a boolean indicating whether the list is empty.
- `get(i)` : Returns the element of the list having index i ; an error condition occurs if i is not in range $[0, \text{size}()-1]$.
- `set(i, e)` : Replaces the element at index i with e , and returns the old element that was replaced; an error condition occurs if i is not in range $[0, \text{size}()-1]$.
- `add(i, e)` : Inserts a new element e into the list so that it has index i , moving all subsequent elements one index later in the list; an error condition occurs if i is not in range $[0, \text{size}())]$.
- `remove(i)` : Removes and returns the element at index i , moving all subsequent elements one index earlier in the list; an error condition occurs if i is not in range $[0, \text{size}()-1]$.

List ADT in Java

```
public interface List <E> extends Iterable<E> {  
    int size();  
    boolean isEmpty();  
    E get(int i) throws IndexOutOfBoundsException;  
    E set(int i, E e) throws IndexOutOfBoundsException;  
    void add(int i, E e) throws IndexOutOfBoundsException;  
    E remove(int i) throws IndexOutOfBoundsException;  
    Iterator<E> iterator();  
}
```

List operation example

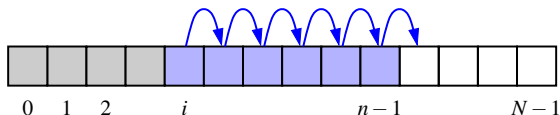
- | | |
|------------|---------|
| ▶ add(0,A) | ▶ (A) |
| ▶ add(0,B) | ▶ (B,A) |
| ▶ get(1) | ▶ A |

List implementation Using Array

- ▶ Array is an obvious choice
- ▶ $A[i]$ stores (a reference to) the element with index i .
- ▶ $\text{get}(i)$: $A[i]$
- ▶ $\text{set}(i, e)$: $A[i]=e$
- ▶ $\text{add}(i,e)$:
 - ▶ we need to make room for the new element by shifting forward

$$A[i], \dots, A[n-1] \quad (1)$$

- ▶ In the worst case ($i = 0$), this takes $O(n)$ time

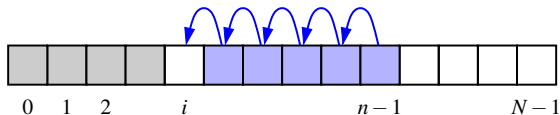


remove(i)

- ▶ need to fill the hole left by the removed element
- ▶ shifting backward the $n - i - 1$ elements

$$A[i + 1], \dots, A[n - 1] \quad (2)$$

- ▶ In the worst case ($i = 0$), this takes $O(n)$ time



Performance of array-based implementation

- ▶ The space used by the data structure is $O(n)$
- ▶ Indexing the element at i takes $O(1)$ time
- ▶ *add* and *remove* run in $O(n)$ time
- ▶ In an add operation, when the array is full, instead of throwing an exception, we can replace the array with a larger one

Method	Running Time
<code>size()</code>	$O(1)$
<code>isEmpty()</code>	$O(1)$
<code>get(i)</code>	$O(1)$
<code>set(i, e)</code>	$O(1)$
<code>add(i, e)</code>	$O(n)$
<code>remove(i)</code>	$O(n)$

List Implementation Using Array

Resize array

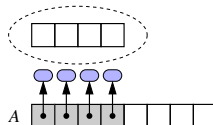
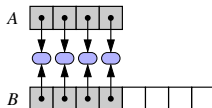
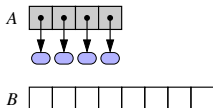
Resizing by copying



- ▶ We can not extend the current array directly
- ▶ The system may allocate neighboring memory locations to store other data,

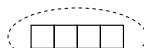
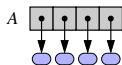
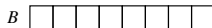
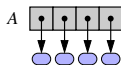
Resize by copying

1. Allocate a new array B with larger capacity.
2. Set $B[k]=A[k]$, for $k=0, \dots, n-1$, where n denotes current number of items.
3. Set $A = B$, that is, we henceforth use the new array to support the list.
4. Insert the new element in the new array.



Resize array in Java

```
private void resize(int capacity) {  
    assert capacity >= n;  
    String[] temp = new String[capacity];  
    for (int i = 0; i < n; i++)  
        temp[i] = items[i];  
    items = temp;  
}
```



What should be the right 'capacity' here?

Resizing strategy

- ▶ When the array is full, we replace the array with a larger one
- ▶ How large should the new array be?
 - ▶ Incremental strategy: increase the size by a constant c
 - ▶ Doubling strategy: double the size
- ▶ Doubling strategy is the choice:

```
public void add(int i, E e) {  
    if (size == data.length) resize(2 * data.length);  
    for (int k=size-1; k >= i; k--)  
        data[k+1] = data[k];  
    data[i] = e;  
    size++;  
}
```

Resize in Stack, Queue etc

- ▶ Resizing is needed in many other data structures
- ▶ Increase the array size when it reaches the end of array.
- ▶ E.g., happens in Stack data structure

```
public void push(String item) {  
    if (n == items.length) resize(2*items.length);  
    items[n++] = item;  
}
```

- ▶ We can also resize into smaller array to free up some memory:

```
public String pop() {  
    if (isEmpty()) throw new NoSuchElementException("Stack  
        underflow");  
    String item = items[n-1];  
    items[n-1] = null;  
    n--;  
    if (n > 0 && n == items.length/4) resize(items.length/2);  
    return item;  
}
```

Amortized algorithm analysis

Consider a dynamic array stack.

- ▶ $push(e)$ will double up the array size if there is not enough space.
- ▶ What is the time complexity of $push(e)$ operation?

Amortized algorithm analysis

Consider a dynamic array stack.

- ▶ $push(e)$ will double up the array size if there is not enough space.
- ▶ What is the time complexity of $push(e)$ operation?
 - ▶ Most of the time it is $O(1)$
 - ▶ Sometimes it copies almost n elements.
- ▶ $push()$ takes *amortized* constant time.

Step-by-step process

Let us count the number of copying operations needed to do a sequence of pushes.

push()	copy	old array size	new array size
1	0	1	-
2	1	1	2
3	2	2	4
4	0	4	-
5	4	4	8
6	0	8	-
7	0	8	-
8	0	8	-
9	8	8	16

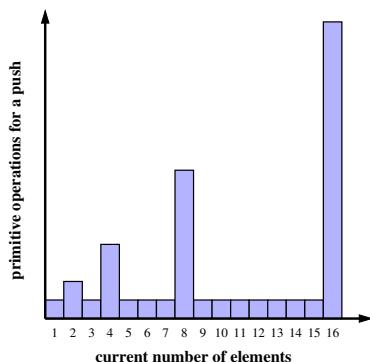
- ▶ 3 pushes requires $2 + 1 = 3$ copies.
- ▶ 5 pushes requires $4 + 2 + 1 = 7$ copies.
- ▶ 9 pushes requires $8 + 4 + 2 + 1 = 15$ copies.
- ▶ $n = 2^k + 1$ pushes requires the following number of copies

$$2^k + 2^{k-1} + \dots + 2^1 + 2^0 = 2^{k+1} - 1 = 2 \times (2^k + 1) - 3 = 2n - 3 \quad (3)$$

- ▶ Asymptotically speaking, the number of copies is about the same as the number of pushes.
- ▶ We say that the algorithm runs at amortized constant time.

Complexity of resizing using Doubling Strategy

Complexity of resizing using Doubling Strategy



Proposition

Let L be an initially empty array list with capacity one, implemented by means of a dynamic array that doubles in size when full. The total time to perform a series of n push operations in L is $O(n)$.

Doubling cost is

$$2^1 + \dots + 2^k$$

Where $2^k \leq n$. It is a geometric sequence with ratio 2.

$$2^1 + \dots + 2^k = 2^{k+1} - 2 \leq 2n - 2$$

Resizing in other geometric sequence

In general

$$\sum_{i=1}^k r^{i-1} = \frac{1}{r-1}(r^k - 1). \quad (4)$$

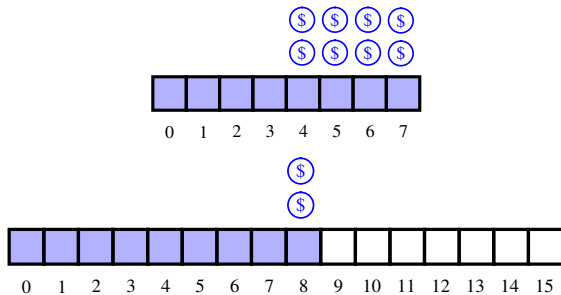
(5)

$$\sum_{i=1}^k 2^{i-1} = \frac{1}{2-1}(2^k - 1) = 2^k - 1. \quad (6)$$

$$\sum_{i=1}^k 3^{i-1} = \frac{1}{3-1}(3^k - 1) = \frac{1}{2}3^k - \frac{1}{2}. \quad (7)$$

In practice, the growth rate is normally smaller than 2.

Why Called Amortized Analysis



Proposition 7.3

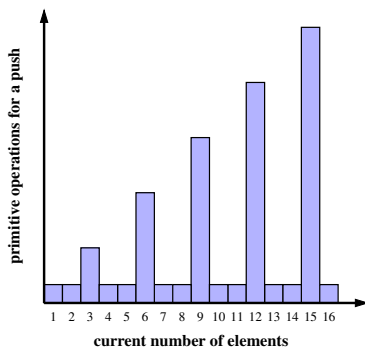
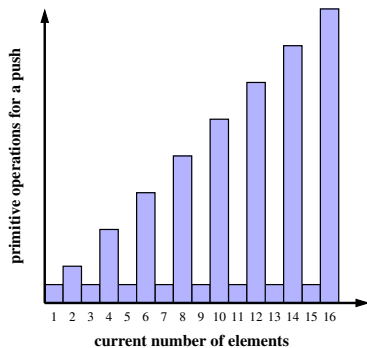
Performing a series of n push operations on an initially empty dynamic array using a fixed increment with each resize takes $\Omega(n^2)$ time.

- ▶ Let $c > 0$ represent the fixed increment in capacity that is used for each resize event.
- ▶ During the series of n push operations, time will have been spent initializing arrays of size $c, 2c, 3c, \dots, mc$ for $m = n/c$,
- ▶ The overall time is proportional to $c + 2c + 3c + \dots + mc$.

$$\sum_{i=1}^m ci = c \sum_{i=1}^m i = c \frac{m(m+1)}{2} \geq \frac{1}{2c} n^2 \quad (8)$$

Arithmetic progression

$$\sum_{i=1}^m ci = c \sum_{i=1}^m i = c \frac{m(m+1)}{2} \geq \frac{1}{2c} n^2 \quad (9)$$



Example: String Concatenation in Lab assignment

```
static String[] readTextBAD(String PATH) throws Exception {  
    BufferedReader br=new BufferedReader(new FileReader(PATH));  
    String text="";  
    String line="";  
    while ((line=br.readLine())!=null) {  
        text=text+" "+line.trim();  
    }  
    String tokens []=text.trim().split("[^a-zA-Z]+");  
    return tokens;  
}
```

- ▶ Each concatenation needs a copying
- ▶ It is similar to arithmetic progression

Takeaways

- ▶ List implemented by Array
- ▶ Resize arrays. Approaches and their complexities
 - ▶ Resizing by doubling the array size. It's time complexity
 - ▶ Resizing by adding a constant. It's time complexity.
- ▶ Amortized algorithm analysis
- ▶ Readings: Goodrich et al. P258-269